

Model Extraction Dominates Static Analysis of Agent Workflows: A Cross-Framework Measurement Study

Anonymous submission

Abstract

Static analysis of agent workflows is only as credible as the behavioral model an extractor recovers from source code: on public agent code, extraction error dominates check error and can reverse both apparent precision and apparent defect prevalence. We make this quantitative with a cross-framework measurement study spanning LangGraph, CrewAI, AutoGen, and Google ADK, mining 922 extracted file-level graphs from public GitHub and validating a 119-graph sample against LLM-reconstructed reference graphs (LLM-assisted exploratory labels, not human-validated ground truth). Measuring a corrected extraction instrument, we find fidelity is the binding constraint: fixing a single conditional-edge extractor bug lifts LangGraph edge recall from 0.68 to 0.82 while leaving the check algorithms untouched. We separate three estimands that a pooled rate conflates: structural-defect prevalence (1 of 119), human-gate policy-violation prevalence (3 of 119), and a heterogeneous composite (4 of 119, reported only as secondary), which post-stratifies to 2.15% under the corpus’s framework mix. We then give the static machinery two defect-independent jobs: a provenance-gated certification that returns `inconclusive` rather than a false `safe` on lossy graphs, and a path-sensitive $\text{graph} \times \text{DFA}$ product that soundly prunes runtime monitors, yielding a small but nonzero gain over alphabet-only pruning. The lesson is methodological: report extraction fidelity alongside every static-analysis claim about agent workflows.

1 Introduction

Large language models are increasingly deployed as autonomous agents that call external tools and APIs (Schick et al. 2023; Yao et al. 2023). Several orchestration frameworks structure this behavior as explicit *workflow graphs*: LangGraph (LangChain 2025), CrewAI (CrewAI 2025), AutoGen (Wu et al. 2024), and Google ADK (Google 2025) each represent computation steps as nodes and allowable transitions as edges. Safety enforcement in these systems remains almost entirely a *runtime* concern: tools such as NeMo Guardrails (NVIDIA 2023) and LlamaGuard (Inan et al. 2023) filter individual calls at execution time, detecting a violation only when the offending path is exercised.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Some defects, however, are topological. Consider an email-triage agent whose developer adds a `draft_response` node but forgets to connect it to `send`: normal-priority emails silently halt at a dead end. A content-level runtime guardrail cannot catch this unless that path runs, but a static analysis of the graph flags the dead end and emits a witness trace. Classical model checkers (Clarke, Grumberg, and Peled 1999; Holzmann 1997; Cimatti et al. 2002) verify such properties in principle, but require hand-translating the system into Promela or SMV—prohibitive for developers iterating on workflows. For conventional static analyzers, decades of deployment experience show that what decides adoption is not detection power but the rate of genuine, actionable findings (Bessey et al. 2010; Johnson et al. 2013; Sadowski et al. 2018). The analogous question for agent workflows—how often these graphs actually contain the defects static checks target, and whether the checks fire on real defects—has received far less measurement.

Thesis. Answering it requires a prior question: *can we trust the graph?* A static check is only as credible as the behavioral model an extractor recovers from source, and on public agent code that extraction is lossy—it manufactures false flags and hides true defects at once. Our central finding is that **extraction error dominates check error** and can reverse both apparent precision and apparent prevalence. We therefore treat static verification of agent workflows as a *measurement* problem. The extraction-and-checking machinery we use, AGENTPROOF, is prior, publicly available infrastructure; this paper’s contribution is not that tool but the cross-framework fidelity measurement it enables and the limits that measurement exposes.

Study. We use AGENTPROOF as a measurement instrument on 922 extracted file-level graphs mined from public GitHub repositories across the four frameworks, and validate a stratified 119-graph sample against *LLM-reconstructed reference graphs* obtained by an independent, adversarially cross-checked LLM pass. These are *LLM-assisted exploratory labels*, not human-validated ground truth; a two-human independent validation protocol (with samples staged) is required future work, and we bound our claims accordingly. Four questions organize the study: how faithfully workflow behavior can be recovered from source, and whether correcting the in-

strument changes the answer (RQ1); what fraction of raised flags is genuine (RQ2); what defect prevalence remains once false negatives are audited, and how distinct estimands differ (RQ3); and whether the static machinery can do a defect-independent job—certifying soundness and pruning runtime monitors—without trusting uncertain extraction (RQ4).

Three estimands, not one prevalence. Because a lossy extractor both manufactures false flags and hides true defects, a single pooled “prevalence” number conflates distinct quantities. We separate three (Section 3, Table 2): flag-level precision (what fraction of raised flags are genuine); *structural-defect* prevalence, a topological property of the graph (1 of 119 sampled graphs); and *human-gate policy-violation* prevalence, a normative judgment that a side-effecting tool is reachable with no human checkpoint (3 of 119). A heterogeneous composite (4 of 119) is reported only as a secondary, validation-sample rate; post-stratified to the corpus’s framework mix it is 2.15%. All rates are small but not the ≈ 0 a naïve reading of flag counts suggests. Three of the four observed cases occur in application-like files, but the sample is too small to establish a stratum difference (Fisher exact $p = 0.10$), so we make no claim that defects concentrate where they matter. Correcting a single conditional-edge extractor bug—not any check algorithm—lifts LangGraph edge recall from 0.68 to 0.82, which is the sense in which fidelity, not the checkers, is the binding constraint.

Contributions.

- **A cross-framework extraction-fidelity measurement** (LLM-assisted) over four frameworks, with a corrected-instrument ablation that isolates extraction error from check error and shows a single extractor fix lifting LangGraph edge recall $0.68 \rightarrow 0.82$ (Section 3).
- **A separation of three estimands** the field tends to pool: flag-level precision, structural-defect prevalence, and human-gate policy-violation prevalence, each with repository-clustered uncertainty and post-stratified to the corpus composition (Section 3, Table 2).
- **A taxonomy of extraction failures** and their downstream effect on findings, diagnosing fidelity—not the check algorithms—as the binding constraint and positioning the study against prior defect-mining work (Section 3, Table 4).
- **A conservative certification gate** that returns *inconclusive* rather than a false *safe* whenever extraction is not certified trace-conservative, plus a path-sensitive graph $\times$ DFA product that soundly prunes runtime monitors with a small but nonzero gain over alphabet-only pruning (Section 4, Table 3).

2 The Agentproof System

Unified graph model. Agentproof represents a workflow as a directed graph $G = (V, E, v_0, V_f)$ with a distinguished entry v_0 and exit set V_f . Each node carries a kind $\kappa_V(v)$ ranging over ENTRY, EXIT, TOOL, LLM, ROUTER, HUMAN, PASSTHROUGH, and SUBGRAPH and, for tool nodes, a set of

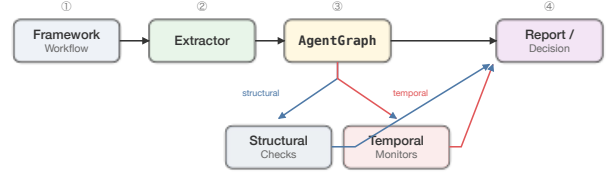


Figure 1: Agentproof extracts a unified graph model from four frameworks, runs structural checks and DFA-compiled temporal policies statically (and at runtime), and emits witness traces.

bound tool names; each edge is DIRECT, CONDITIONAL, PARALLEL, or LOOP. Every node and edge additionally carries extraction *provenance*—an origin (runtime object, explicit AST, inferred, synthesized, unknown), a confidence (exact, may, heuristic), and a source span—so that structure read from a framework declaration is distinguishable from structure the extractor guessed, and downstream witness traces inherit the weakest confidence they traverse. Per-framework extractors bridge four heterogeneous representations—each with different entry/exit conventions, edge semantics, and hierarchy models—into this single typed model. The extractors themselves encode framework semantics and are handwritten once per framework; what is eliminated is the per-workflow manual translation that SPIN or NuSMV require. Extraction has two paths: a runtime extractor that reads the compiled graph object, and a static AST fallback (used for large-scale mining, where installing every repository’s dependencies is infeasible).

An orthogonal attribute ontology. The single-label kinds above are a compatibility view for cross-framework comparison, but forcing each element into one bucket mislabels multi-faceted nodes—an LLM node that also invokes a tool, or a conditional edge that also closes a loop—and was itself a source of extraction error (one reference graph recorded a conditional loop as a bare LOOP). We therefore layer a non-exclusive ontology over κ_V/κ_E : each node carries an *effect* set (read, write, execute, communicate, financial, delete) and a *capability* set (llm, invokes_tool, human_pause, routes, state_update, subgraph), and each edge a *back-edge* flag, so a conditional loop is represented as (CONDITIONAL, back_edge) rather than collapsed to a single LOOP label. These descriptors are orthogonal—a node may read, write, and communicate at once—and are the ontology we treat as primary; the exclusive kinds are retained only for legacy checks and comparison. The full vocabulary and its per-framework mapping appear in the supplement.

Structural checks. Six checks run over the topology, each in $O(|V| + |E|)$ and each emitting a *witness trace* on failure. *Exit reachability* verifies $V_f \subseteq \text{Reach}(v_0)$; *reverse reachability* flags reachable nodes with no path to any exit—executions entering such a node can never complete (passing this check does not guarantee termination, which depends on runtime routing); *dead-end detection* flags non-exit nodes

with no successor; *router shape* is a framework-convention check requiring router outgoing edges to be conditional; *human presence* flags workflows with no HUMAN node (a deliberately blunt policy check we evaluate against a risk-aware variant); and *tool declaration* flags tool nodes with an empty tool set. A seventh, path-sensitive check, *human-gate coverage*, is the risk-aware variant: it flags a workflow only if a *sensitive* tool is reachable from v_0 without passing a HUMAN node (Section 3). Witness traces follow standard model-checking practice (Clarke, Grumberg, and Peled 1999) and pinpoint the offending path.

Temporal policies with LTL_f semantics. Beyond topology, Agentproof supports a policy DSL over finite traces (LTL_f (De Giacomo and Vardi 2013)): a workflow execution is a finite event sequence, and each of the seven DSL forms denotes an LTL_f formula—*forbidden* ($G \neg a$), *response* ($G(a \rightarrow F b)$), *strong until* ($a U b$, including nested until), *bounded response* ($G(a \rightarrow F_{\leq k} b)$), *response chain* ($G(a \rightarrow F(b \wedge Fc))$), *conjunction*, and *disjunction*. Formulas are parsed into a typed recursive AST and compiled compositionally to a DFA by LTL_f formula progression (2–3 states for base patterns, $k+2$ for bounded response), with two disjoint violation mechanisms: *violation states* are exactly the doomed states—those from which no accepting state is reachable—so a bad prefix is flagged at the earliest event that makes every extension violating; and a trace ending in a non-accepting state—a pending F -obligation—violates the property at termination. The compiler’s verdicts are differentially tested against an independent, directly recursive reference evaluator, exhaustively over all traces up to length six for two-atom formulas and property-based beyond. Runtime monitors evaluate events in $O(1)$ each and apply the end-of-trace acceptance check when the workflow terminates; obligations still open when a run is aborted are reported as inconclusive rather than violated, following three-valued runtime verification (Bauer, Leucker, and Schallhart 2011).

Static temporal verification. Policies are checked statically via a graph $\times$ DFA product: BFS from (v_0, q_0) advances the DFA at each node and follows graph edges. Executions are modeled as finite *maximal* paths, judged by LTL_f : paths ending at an exit node or at a dead end. Because static extraction cannot prove which of a tool node’s declared tools will be invoked, or whether any will be, the product is *conservative*: at each tool node it closes the DFA state over every finite invocation sequence drawn from the declared tools, including the empty one—a false alarm is acceptable, a false proof is not. The checker returns one of three verdicts: *may-violate*, when some resolution reaches a DFA violation state (bad prefix) or a termination point with a pending obligation; *safe*, when no resolution does; and *inconclusive*, when the only finding is a reachable product cycle on which an obligation could be postponed forever, or no termination point is reachable. Universal satisfaction over the finite terminating traces is itself decidable; treating a divergent cycle as inconclusive, rather than enumerating the terminating traces that thread it, is a deliberate engineering-scope choice, not a semantic limitation. Witness paths carry the provenance-

derived confidence of the edges and bindings they traverse. Explored states are bounded by $|V| \cdot |Q|$, so verification is linear in graph size for fixed-size DFAs.

Soundness gated on certified extraction. Soundness is a claim about *event* traces, not graph paths. Let $\text{Traces}_{ev}(P)$ be the concrete finite terminating event traces a workflow P can emit (tool identities and arguments, node-emitted events, termination), and let λ label each event, inducing a graph event-language $\mathcal{L}(G, \lambda)$. If $\text{Traces}_{ev}(P) \subseteq \mathcal{L}(G, \lambda)$ (*event-trace conservatism*) and every trace in $\mathcal{L}(G, \lambda)$ is accepted by the policy DFA, then every concrete finite terminating execution of P satisfies the policy—the guarantee we exploit for monitor pruning (Section 4); the full proof is in the supplement. Graph-path containment alone does *not* suffice: a node may issue tool calls the graph never records, so a path-conservative graph can still omit the events that decide the property. Because a lossy extractor cannot be assumed event-trace conservative, the checker gates on certification. `check_temporal_property` returns *safe* only when the traversed region is *certified*—every traversed edge and every visited tool-node binding carries exact provenance, or the caller asserts an authored graph (`assume_trace_conservative`)—and it stamps the result with a certified key; otherwise it downgrades to *inconclusive* with reason `uncertified_extraction`. A `may_violate` finding is *never* suppressed by the gate. This is the may/must-lite discipline: provenance now gates soundness itself, not merely the confidence annotation on a witness. Authored corpus graphs and runtime-extracted graphs (absent dynamic graph modification) certify; the lossy AST fallback used for mining (edge recall 0.65, Section 3) rarely does, so on mined graphs the gate withholds *safe* rather than emit a false proof, and we report only measurements—the *certified* field making the distinction machine-checkable.

3 Base-Rate Study on Public GitHub Workflows

To estimate defect base rates on independently authored workflows—and to measure extractor fidelity on code we did not write—we mined public GitHub repositories across all four frameworks. We state the population precisely: *public GitHub files matching framework-specific code searches*, not production deployments. The unit of analysis is one source file mapped to one *extracted file-level graph*; some files hold multiple executable roots, only task declarations, or fixtures, so a fully executable-workflow-root unit is future work. We census the population below and report all prevalence figures as *LLM-assisted exploratory estimates*, not ground truth.

Mining and corpus census. Using GitHub code search (gh CLI, July 2026, newest pinned commit 2026-07-11, with per-framework query sets and per-repository caps) we streamed each candidate repository through clone, extraction, and cleanup. After removing one repository of our own test fixtures (8 files; a contamination detected during revision and excluded from every number), the corpus is **922 extracted file-level graphs** from 252 distinct repositories.

Table 1: Extractor fidelity on real workflows: the corrected instrument (v2, primary; $n=115$) with the as-mined instrument’s edge recall (v1, $n=119$) as an ablation column. Node detection is strong and stable (v2 overall $P=0.89$, $R=0.85$); edge recall and node-kind accuracy are the framework-dependent bottleneck. Repository-clustered bootstrap CIs and the full per-framework v1 breakdown are in the supplement.

Framework	n	Edge P	Edge R	Kind	Edge R_{v1}
LangGraph	40	0.931	0.819	0.674	0.682
CrewAI	20	0.700	0.692	0.929	0.692
AutoGen	32	0.906	0.730	0.897	0.770
ADK	23	0.395	0.355	0.938	0.382
Overall	115	0.777	0.679	0.833	0.649

ries (LangGraph 400, AutoGen 359, CrewAI 113, ADK 50), with provenance recorded as repository, commit SHA, and file path. Mined graphs are small—median 2 non-sentinel nodes (mean 3.9)—and predominantly static linear pipelines. The corpus is *not* mostly applications: 32% of files carry tutorial/test/demo-style names, 48 topology groups are topologically identical duplicates across repositories (170 redundant graphs, retained but reported), and a source-level classification of the validation sample (below) found 44% tutorial or course material, 17% toy demos, 7% test files, and 33% application-like code. We therefore report prevalence both pooled and stratified by this classification.

Reference graphs and validation. On a stratified 119-graph sample from 87 repositories (LangGraph 40, CrewAI 20, AutoGen 35, ADK 24) we ran two independent LLM-agent passes (Claude Opus 4.8 under a multi-agent harness; all outputs released). First, an agent reconstructed a reference graph from source alone, blind to the extractor’s output and following its node-id conventions; we score node/edge precision–recall and node-kind accuracy against these *LLM-reconstructed reference graphs* (reconstruction confidence: 77 high, 33 medium, 9 low). Second, every flag was triaged from source (REAL/EXTRACTION-ARTIFACT/INTENTIONAL) and an adversarial verifier attempted to overturn each label (4.6% overturned; unresolved labels recorded as ARGUABLE). We state the limitation squarely: reference graphs are LLM reconstructions, not human annotations, and both passes share a model family. Results are stable on the high-confidence subset (edge recall 0.67 vs. 0.65 overall), every triage label carries a source-line justification that survived an adversarial pass, and a revision re-audit overturned one label of 16 re-examined. **Two-annotator independent human validation was not performed; it is required future work, and the protocol and samples are staged.** We report Wilson 95% CIs for proportions and repository-clustered bootstrap CIs (10,000 resamples) for means, since workflows from one repository are not independent.

Extraction fidelity is the bottleneck. Node detection stays strong and stable across instruments (v2 overall $P=0.89$,

$R=0.85$; v1 0.91/0.85), but edge recall and node-kind accuracy are the bottleneck and are strongly framework-dependent (Table 1): the corrected instrument recovers only 0.36 of ADK’s edges—whose nested *SequentialAgent/ParallelAgent* composition, and AutoGen’s participant lists with dynamic speaker selection, encode topology *implicitly*—versus 0.82 on LangGraph, whose edges are explicit. On LangGraph the dominant remaining error is instead node-kind (`tool`→`llm`: nodes invoking tools in their body with no declared binding; kind accuracy 0.67). This is a property of the static AST fallback used at scale, not of the runtime extractors that read the compiled framework object (for ADK the runtime graph is far richer, 16 vs. ~ 5 edges); realizing that ceiling would require sandboxed execution of untrusted code, which we do not attempt. A post-hoc audit demonstrates the diagnosis *causally*: the extractor silently dropped conditional edges declared through LangGraph’s common three-argument `add_conditional_edges` form. Fixing that one bug and re-extracting at the identical pinned commits—the check algorithms untouched—raises LangGraph edge recall from 0.68 to 0.82 (the v1→v2 shift in Table 1) and removes roughly a quarter of the structural flags (314→239 flagged workflows; unreachable-exit flags 86→32), directly confirming extraction, not the checks, as the bottleneck. Both instruments are released.

Flag precision on extracted graphs. Distinct from prevalence, we ask what fraction of the checker’s *flags* on extracted graphs are genuine. The as-mined study raised 186 flags on 114 file-level graphs; adversarial triage confirmed 2 as genuine (both missing human gates on sensitive workflows), classifying 42% as extraction artifacts and 50% as intentional design (full triage breakdown in the supplement). This is a workflow-level PPV of $2/114 = 1.75\%$ [0.48, 6.17]% (repository-clustered bootstrap [0.00, 4.59]%) and a flag-level confirmed PPV of $2/186 = 1.08\%$ [0.30, 3.84]%. Because flags are correlated—one missing edge fires several structural checks—we also dedup to root cause (186 → 161 flags, 25 structural collapsed; $2/161 = 1.24\%$ confirmed). Counting all 12 ARGUABLE labels pessimistically as genuine bounds the flag-level rate at $14/186 = 7.53\%$ [4.54, 12.24]% ($14/161 = 8.70\%$ deduped), so the identification rate lies in [1.08%, 7.53%]. *None of the 72 structural flags was confirmed genuine* (0 genuine, 1 arguable; CI [0, 5.1]%), and structural flags are almost entirely a LangGraph phenomenon—partly *by construction*, since the other three extractors emit connected topologies and cannot produce a dead end or unreachable exit. This as-mined 186-flag universe (genuine=2) is *distinct* from the reconstructed-graph re-run that yields the four confirmed estimand cases below (one of them structural); we never pool the two.

Defining a defect. Flag triage cannot see false negatives: a defect the lossy extractor never surfaces is absent from the flag universe entirely. We therefore re-ran every check on the 119 LLM-reconstructed reference graphs—which can express dead ends and unreachable exits for all four frameworks—and adversarially triaged the results, recovering two genuine defects the extracted study missed. We separate two kinds of finding the pooled count conflated.

Table 2: Three defect estimands on the 119-graph validation sample (LLM-assisted exploratory estimates), with Wilson 95% CIs (%) and post-stratified corpus estimates (repository-clustered bootstrap). The composite unions a structural graph property with a policy judgment and is reported only as a heterogeneous secondary summary.

Estimand	Sample	Wilson 95%	Post-strat.
Structural defect	1/119 (0.84%)	[0.15, 4.61]	0.23% [0.00, 0.80]
Human-gate policy	3/119 (2.52%)	[0.86, 7.15]	1.92% [0.00, 4.85]
Composite [†]	4/119 (3.36%)	[1.31, 8.32]	2.15% [0.20, 5.12]

[†]Heterogeneous secondary summary (union of the two primary estimands).

A *structural defect* is a genuine topology error (unreachable exit, dead end, mis-wired router)—a property of the graph, confirmed on the reference graph. A *human-gate policy violation* is a normative judgment, not a structural fact: a side-effecting or irreversible tool (SQL execution, payment, external write) is reachable along some path with no intervening HUMAN checkpoint, judged against an explicit policy—*no irreversible tool without a preceding human gate*—and it would not appear under a permissive policy. We report each as a **distinct estimand** and their union (*composite*) only as a heterogeneous secondary summary.

Three estimands and post-stratification. Table 2 reports all three. The one structural defect is a router mis-wiring in an ADK SQL agent (application-like): 1/119 [0.15, 4.61]%. The three policy violations are a LangGraph chatbot whose SQL-executing tool is reachable with no human gate, and a CrewAI workflow and an ADK payment agent flagged for missing human presence on side-effecting paths: 3/119 [0.86, 7.15]%. The composite is 4/119 [1.31, 8.32]%. The sample over-represents ADK—20% of the sample but 5.4% of the corpus (50/922)—and ADK carries 2 of the 4 cases (per-framework composite: LangGraph 1/40, CrewAI 1/20, AutoGen 0/35, ADK 2/24), so the unweighted rate overstates. Re-weighting each stratum to its corpus share (repository-clustered bootstrap) lowers every estimate: structural 0.23% [0.00, 0.80], policy 1.92% [0.00, 4.85], composite **2.15%** [0.20, 5.12], against the unweighted validation-sample composite of 3.36%. We treat the post-stratified figures as the corpus-facing estimate and 3.36% as a validation-sample statistic only. Three of four observed cases occurred in application-like files ($3/39 = 7.7\%$ vs. $1/80 = 1.3\%$ in tutorials, demos, and tests), but the sample is too small to establish a stratum difference (Fisher exact $p=0.10$; OR 6.58 [0.66, 65.5], the interval spanning 1); we report the association descriptively only and make no claim that defects concentrate where the stakes are highest.

Risk-aware gating. The blunt human-presence check fires on 91% of mined file-level graphs with only 2 of 113 firings genuine—uninformative as a detector, motivating the risk-aware *human-gate coverage* check: flag only when a sensitive tool (destructive/write/exec/communication/financial, by a keyword lexicon over bound tool names) is reachable without a HUMAN gate. On the 922 mined graphs it fires

zero times—a statement about extracted *declarations*, not the code: no mined graph declares a sensitive binding, while source-level classification finds side-effecting tools in 32 of 119 sampled workflows, invisible to the extractor through the `tool→llm` kind error. On the reconstructed graphs, where bindings are recovered from source, the same check fires three times and one firing is the genuine SQL-gate defect above. The lexicon is *correct relative to its declared-tool abstraction*; its recall is bounded by extraction fidelity—the paper’s central diagnosis.

Threats to validity. *Construct:* the composite estimand unions a structural graph property with a policy judgment, which is why we report the two separately. *Internal:* reference graphs and triage labels come from LLM passes sharing a model family, so correlated errors are possible; we report reconstruction confidence, adversarial overturn rates, and a re-audit, but these are not independent human judgments. *External:* the population is public GitHub files, not production deployments, and the unit is a file-level graph, so results may not transfer to executable workflow roots or private code. *Sampling:* the validation sample is stratified and over-represents ADK; we post-stratify to corpus composition, but small per-stratum counts leave wide intervals. *Label reliability:* no human validation has yet been performed; a two-annotator independent protocol with staged samples is required future work. Accordingly we frame every prevalence number as an LLM-assisted exploratory estimate, not ground truth.

4 Controlled Evaluation and Monitor Selection

Controlled detection test. A curated corpus of 18 workflows spanning all four frameworks, seeded with known anti-patterns, confirms the checks fire when defects are genuinely present: Agentproof detects every injected structural defect and, with a risk-aware sensitive-tool set, flags 8 workflows whose sensitive path (e.g. `account_create`, `publish`, `auto_remediate`) is reachable without a human gate, versus 10 for the blunt human-presence check—declining to flag two workflows that lack a human node but perform no sensitive action. This corpus is a controlled test that the algorithms work, not evidence of prevalence. Verification is sub-second even for synthetic graphs of 5,000 nodes (far larger than any observed workflow); details, per-framework extractor accuracy on stubs, and a Promela/SMV modeling-effort comparison are in the supplementary material.

Static monitor selection is an optimization, not a headline.

The graph $\times$ DFA product gives the static machinery a job that does not depend on defects existing: deciding which runtime monitors to deploy. Its value is *soundness*, not scale, and we treat the pruning layer and its policy DSL as supporting infrastructure for the certification gate of Section 2, which returns *safe* for a monitor only when the traversed graph region is certified trace-conservative. A policy whose product is *safe*—no resolution of the workflow’s declared tool bindings reaches a bad prefix or a termination point with a pending obligation—can never fire at runtime, so its mon-

itor need not be deployed; violations (`may_violate`) are never suppressed. On the curated corpus (exact-provenance authored graphs) the gate certifies 237 of 270 monitor instances (87.8%) as provably inert and declines 14 (all on one seeded-livelock workflow) as *inconclusive*, leaving the remaining 12.2% (19 that may fire) to runtime enforcement. Returning *inconclusive* on a workflow whose cycles could postpone an obligation forever is an engineering scope choice, not undecidability: universal satisfaction over finite terminating traces is decidable, and we simply do not bound the divergent case.

Descriptive vs. sound pruning on mined code. This certified rate does not survive contact with lossy extraction, and stating so removes a latent contradiction. What an earlier draft reported as $\sim 93\%$ “pruning” on mined graphs is really pure *alphabet-inertness*: all 13,950 mined (workflow, policy) instances are alphabet-inert (100%) because no mined workflow invokes any of the 15 policies’ tools—a descriptive property of a mismatched vocabulary, not a sound prune. On a may-provenance graph a missing edge could hide exactly the tool that makes a live monitor look inert. The sound gate therefore certifies *none* of the 13,950 mined instances—every mined graph carries only may-provenance nodes and edges, so the gate refuses them all as uncertified extraction—reconciling this section with the paper’s own claim that pruning cannot soundly apply to lossy graphs.

Does path-sensitivity buy anything? Alphabet-inertness needs no product; the product earns its keep only when a policy’s atoms actually occur and inertness turns on *which* paths reach them. We built 18 (workflow, policy) pairs of exactly this kind and compared four selection strategies (Table 3). A runtime-only strategy—prune a monitor if no observed execution triggers it—certifies 0, since one unobserved path forbids the claim. Alphabet-only prunes 1, node-reachability 3, and the path-sensitive product 10: 9 more inert monitors than the symbol-set check and 7 more than node reachability, with *no* false prunes across 50 concrete executions. We are deliberately conservative about what this shows. Five of the 9 incremental cases are linear curated response-obligation pipelines in which the consequent trivially dominates every path; the genuine evidence of path-sensitivity on branching and looping topologies rests on the 4 purpose-built *synthetic* fixtures (labeled synthetic, not sampled)—including a declared-but-unreachable forbidden tool and an obligation that is bounded across every branch. The contribution is thus a sound, cheap optimization with a small but real path-sensitive gain, not a scale result.

5 Related Work

Runtime agent safety. NeMo Guardrails (NVIDIA 2023), Guardrails AI (Guardrails AI 2023), LlamaGuard (Inan et al. 2023), and Rebuff (Protectai 2023) enforce safety at the point of LLM output or tool call, catching content-level violations only when the offending path runs. Sandboxed and adversarial evaluations such as ToolEmu (Ruan et al. 2024) and AgentDojo (Debenedetti et al. 2024) measure agent risk by executing behaviors. Agentproof is complementary to both:

Table 3: Monitor pruning on 18 (workflow, policy) pairs whose policy atoms occur in the workflow. The path-sensitive product certifies 9 more inert monitors than a symbol-set check and 7 more than node reachability, with no false prunes across 50 concrete executions.

Strategy	Pruned (/18)	Δ vs. alphabet
Runtime-only (execution witness)	0	—
Alphabet-only (symbol set)	1	—
Node-reachability	3	+2
Path-sensitive product	10	+9

it analyzes structural properties of the topology before deployment, and its monitor-selection layer (Section 4) decides which runtime monitors are even needed.

Concurrent agent-policy systems. A concurrent line of work enforces policies at the tool-call boundary with formal machinery: Agent-C compiles temporal safety specifications to constraints checked at generation time with conformance guarantees (Kamath et al. 2026); AgentSpec provides a customizable runtime-enforcement DSL for safe agent execution (Wang, Poskitt, and Sun 2025); ClawGuard and SecureClaw interpose rule sets and authorization boundaries against prompt injection (Zhao et al. 2026; Ma and Schmid 2026); PolicyGuard trains a compact guardrail model for trajectory-level policy violation detection (Wen et al. 2025); and AgentFlow builds cross-framework agent dependency graphs for static security analysis at scale (Wang et al. 2026). These systems address *how* to enforce or analyze; our study measures *when* the static side of that stack pays off—the base rate of the defects such tools target in public code, and where the error budget actually sits (model extraction). That measurement is orthogonal to, and needed by, every system above.

Empirical characterizations of agent defects. Recent measurement studies catalogue where agent systems break. AgentAble derives a defect taxonomy and detector from developer discussions (Ning et al. 2024); a characterization study of orchestration-framework bugs identifies nine root causes across roughly 1,026 bugs (Xue et al. 2025); and a security-debt study reports 221 vulnerabilities spanning fourteen types in agent applications (Shen et al. 2025). These establish that defects exist and how they distribute by category. Our study is complementary but asks a different question: not *which* defects occur, but *how reliably a static analyzer can recover the behavioral model it needs to find them*, and what the base rate of a checkable topology-level defect class is once instrument error is separated from signal. Table 4 places these efforts along six axes; the feature that distinguishes ours is a cross-framework measurement of extraction fidelity, an explicit separation of estimands, and a provenance-gated soundness guarantee.

Model checking and workflow soundness. Classical model checkers—SPIN (Holzmann 1997), NuSMV (Cimatti et al. 2002), CBMC (Kroening and Tautschnig 2014)—verify the same class of properties but require manual transla-

Table 4: Positioning against measurement and enforcement work for LLM agents. Prior measurement studies characterize *which* defects occur; enforcement systems address *how* to constrain agents at runtime. Ours measures the fidelity of the extracted model on which static checks depend, separates distinct prevalence estimands, and gates soundness on extraction provenance.

Work	Corpus	Frameworks	Model	Properties	Validation	Guarantees
Ning et al. (2024)	dev discussions	agnostic	—	defect taxonomy	manual	none
Xue et al. (2025)	~1,026 bugs	orchestration	—	9 root causes	manual	none
Shen et al. (2025)	221 vulns	agent apps	—	14 vuln. types	manual	none
Wang et al. (2026)	5,399 progs	cross-framework	dep. graph	security patterns	—	none (unvalidated)
Wang, Poskitt, and Sun (2025)	case studies	runtime	rule DSL	safety rules	case studies	runtime enforcement
Kamath et al. (2026)	case studies	generation-time	temporal aut.	temporal constraints	conformance	conformance (runtime)
Ours	922 graphs, 252 repos	4 (cross)	graph + LTL _f	struct. + policy estimands	LLM-recon.; human staged	provenance-gated soundness

tion into Promela/SMV/annotated C; Agentproof extracts the model automatically. Our policy semantics follow LTL on finite traces (De Giacomo and Vardi 2013), with monitoring in the style of JavaMOP (Jin et al. 2012) and three-valued LTL runtime verification (Bauer, Leucker, and Schallhart 2011). Business-process soundness for Petri nets (van der Aalst 2011) and BPMN (Dijkman, Dumas, and Ouyang 2008) studies analogous reachability properties, but targets visual notations rather than the programmatic API objects agent frameworks expose.

Base rates for static analysis. Industrial experience with conventional static analyzers established that false-positive and actionability rates, not detector cleverness, decide adoption: Coverity’s deployment history (Bessey et al. 2010), developer studies of why warnings are ignored (Johnson et al. 2013), and Google’s Tricorder lessons (Sadowski et al. 2018). Our study is the agent-workflow instance of that genre—and adds a failure mode those studies did not face: for agent frameworks the *model extraction* step itself dominates the error budget, so that flag counts on extracted graphs track extractor fidelity more than any underlying defect rate (Section 4).

Agent safety and evaluation. Work on agent safety emphasizes alignment, capability evaluation, and governance (Anthropic 2023; METR 2024; Gabriel et al. 2024). Our contribution is orthogonal and empirical: base-rate measurements of a specific, checkable defect class in public agent workflows, with the estimands and their instrument error separated.

6 Conclusion

Static checks on agent workflows are only as credible as the behavioral model they run on, and recovering that model from public agent code—not the check algorithms—is the binding constraint. On a cross-framework corpus of 922 extracted file-level graphs from 252 public GitHub repositories, extraction error dominates check error and can reverse both apparent precision and apparent prevalence; the DSL and monitor-pruning layer are supporting infrastructure for that finding, not the headline.

The estimates are textured, not a null. Read as distinct estimands rather than a pooled rate, LLM-assisted exploratory estimates put confirmed structural defects at 0.84% and human-gate policy violations at 2.5% of the validation sample (Table 2), with the composite post-stratifying

to 2.2%; three of four observed cases occur in application-like files, too few to establish a stratum difference (Fisher exact $p=0.10$). Yet flag-level precision stays low (workflow-level PPV 1.8%), none of the structural flags on extracted graphs was a confirmed defect, and the diagnosis holds across every cut—edge recall 0.68 on real code, 0.36 on implicit-topology ADK. The static machinery still keeps a defect-independent role: under certified event-trace containment the graph $\times$ DFA product soundly prunes 237 of 270 curated monitor instances (88%; Table 3), a monitor-selection layer in a static-plus-runtime safety stack rather than a scale result.

Limitations. The population is public GitHub, not production systems; two thirds of it is tutorial-grade, which we census and stratify but cannot fully correct. The unit of analysis is one extracted file-level graph, not an executable workflow root; a root-level unit is future work. Reference graphs and triage labels are LLM reconstructions with adversarial verification, not human annotations; confidence-stratified sensitivity checks and released per-label justifications mitigate but do not eliminate this, and a two-human independent validation is staged (protocol and samples prepared) as the immediate next step. Prevalence CIs are wide at $n=119$; the estimates are best read as “rare, but real”, not as precise rates. We read these results as a redirection—toward higher-fidelity extraction and per-workflow policy instantiation.

Artifact. An anonymized artifact is submitted with this paper, not merely promised: the tool, both corpora, the mining and validation pipeline, the LLM-reconstructed reference graphs, triage labels with per-label justifications, and all evaluation scripts, packaged for double-blind review under the MIT license.

Ethical Statement

The study mines only *public* GitHub repositories, records exact provenance (repository at commit SHA plus file path), and redistributes only derived graph abstractions, never third-party source; no personal data is processed. Confirmed defects are reported without identifying maintainers, and we disclose them to the affected repositories before publication. The intended impact is defensive—the honest finding that such defects are rare in public code should calibrate, not inflate, claims about static verification’s role in agent safety.

References

- Anthropic. 2023. Anthropic’s Responsible Scaling Policy. <https://www.anthropic.com/responsible-scaling-policy>. Accessed: 2026-03-01.
- Bauer, A.; Leucker, M.; and Schallhart, C. 2011. Runtime Verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4).
- Bessey, A.; Block, K.; Chelf, B.; Chou, A.; Fulton, B.; Hallem, S.; Henri-Gros, C.; Kamsky, A.; McPeak, S.; and Engler, D. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Communications of the ACM*, 53(2): 66–75.
- Cimatti, A.; Clarke, E.; Giunchiglia, E.; et al. 2002. NuSMV 2: An OpenSource Tool for Symbolic Model Checking. In *International Conference on Computer Aided Verification (CAV)*. Springer.
- Clarke, E. M.; Grumberg, O.; and Peled, D. A. 1999. *Model Checking*. MIT Press.
- CrewAI. 2025. CrewAI documentation. <https://docs.crewai.com>. Accessed: 2026-02-28.
- De Giacomo, G.; and Vardi, M. Y. 2013. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In *International Joint Conference on Artificial Intelligence (IJCAI)*.
- Debenedetti, E.; Zhang, J.; Balunović, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- Dijkman, R. M.; Dumas, M.; and Ouyang, C. 2008. Semantics and analysis of business process models in BPMN. *Information and Software Technology*, 50(12): 1281–1294.
- Gabriel, I.; et al. 2024. The Ethics of Advanced AI Assistants. *arXiv preprint arXiv:2404.16244*.
- Google. 2025. Google Agent Development Kit (ADK) documentation. <https://google.github.io/adk-docs/>. Accessed: 2026-02-28.
- Guardrails AI. 2023. Guardrails AI: Adding Guardrails to Large Language Models. <https://github.com/guardrails-ai/guardrails>. Accessed: 2026-03-01.
- Holzmann, G. J. 1997. The Model Checker SPIN. *IEEE Transactions on Software Engineering*, 23(5): 279–295.
- Inan, H.; Upasani, K.; Chi, J.; et al. 2023. LlamaGuard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674*.
- Jin, D.; Meredith, P. O.; Lee, C.; and Roşu, G. 2012. JavaMOP: Efficient Parametric Runtime Monitoring Framework. In *International Conference on Software Engineering (ICSE)*.
- Johnson, B.; Song, Y.; Murphy-Hill, E.; and Bowdidge, R. 2013. Why Don’t Software Developers Use Static Analysis Tools to Find Bugs? In *International Conference on Software Engineering (ICSE)*.
- Kamath, A.; Zhang, S.; Xu, C.; Ugare, S.; Singh, G.; and Misailovic, S. 2026. Enforcing Temporal Constraints for LLM Agents. Preprint. <https://github.com/structuredllm/agent-c>.
- Kroening, D.; and Tautschnig, M. 2014. CBMC – C Bounded Model Checker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer.
- LangChain. 2025. LangGraph documentation. <https://docs.langchain.com/oss/python/langgraph/overview>. Accessed: 2026-02-28.
- Ma, Y.; and Schmid, S. 2026. SecureClaw: Clawing Back Control of LLM Agents. *arXiv:2606.09549*.
- METR. 2024. METR: Model Evaluation and Threat Research. <https://metr.org>. Accessed: 2026-03-01.
- Ning, K.; Chen, J.; Zhang, J.; Li, W.; Wang, Z.; Feng, Y.; Zhang, W.; and Zheng, Z. 2024. Defining and Detecting the Defects of the Large Language Model-based Autonomous Agents. *arXiv preprint arXiv:2412.18371*.
- NVIDIA. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. <https://github.com/NVIDIA/NeMo-Guardrails>. Accessed: 2026-03-01.
- Protectai. 2023. Rebuff: Self-hardening Prompt Injection Detector. <https://github.com/protectai/rebuff>. Accessed: 2026-03-01.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations (ICLR)*.
- Sadowski, C.; Aftandilian, E.; Eagle, A.; Miller-Cushon, L.; and Jaspan, C. 2018. Lessons from Building Static Analysis Tools at Google. *Communications of the ACM*, 61(4): 58–66.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.
- Shen, Z.; Dai, J.; Zhang, Y.; and Yang, M. 2025. Security Debt in LLM Agent Applications: A Measurement Study of Vulnerabilities and Mitigation Trade-offs. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- van der Aalst, W. M. P. 2011. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer.
- Wang, H.; Poskitt, C. M.; and Sun, J. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. *arXiv preprint arXiv:2503.18666*.
- Wang, S.; Hou, X.; Zhao, Y.; Cheng, X.; and Wang, H. 2026. AgentFlow: Building Agent Dependency Graphs for Static Analysis of Agent Programs. *arXiv:2607.01640*.
- Wen, X.; Mo, W. J.; Xie, Y.; Qi, P.; and Chen, M. 2025. Towards Policy-Compliant Agents: Learning Efficient Guardrails for Policy Violation Detection. *arXiv:2510.03485*.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM)*.

Xue, Z.; Zhao, Y.; Wang, S.; Chen, K.; and Wang, H. 2025. A Characterization Study of Bugs in LLM Agent Workflow Orchestration Frameworks. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

Zhao, W.; Li, Z.; Zhang, P.; and Sun, J. 2026. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. arXiv:2604.11790.

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this .tex file directly.

For each question (that applies), replace the “Type your response here” text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel  
claims are included}  
{ (yes/partial/no) }  
Type your response here
```

you would change it to:

```
\question{Proofs of all novel  
claims are included}  
{ (yes/partial/no) }  
yes
```

Please make sure to:

- Replace **ONLY** the “Type your response here” text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the `\question` command or any other lines in this document.

You can `\input` this .tex file right before `\end{document}` of your main file or compile it as a stand-alone document. Check the instructions on your conference’s website to see if you will be asked to provide this checklist with your paper or separately.

1. General Paper Structure

- 1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) **yes**
- 1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) **yes**
- 1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) **yes**

2. Theoretical Contributions

- 2.1. Does this paper make theoretical contributions? (yes/no) **no**

If yes, please address the following points:

- 2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) **NA**
- 2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) **NA**
- 2.4. Proofs of all novel claims are included (yes/partial/no) **NA**
- 2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) **NA**
- 2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) **NA**
- 2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) **NA**
- 2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) **NA**

3. Dataset Usage

- 3.1. Does this paper rely on one or more datasets? (yes/no) **yes**

If yes, please address the following points:

- 3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) **yes**
- 3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) **partial**
- 3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) **yes**
- 3.5. All datasets drawn from the existing literature (potentially including authors’ own previously published work) are accompanied by appropriate cita-

tions (yes/no/NA) [NA](#)

- 3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) [NA](#)
- 3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisfying (yes/partial/no/NA) [NA](#)

4. Computational Experiments

- 4.1. Does this paper include computational experiments? (yes/no) [yes](#)

If yes, please address the following points:

- 4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) [NA](#)
- 4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) [yes](#)
- 4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) [yes](#)
- 4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) [yes](#)
- 4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) [partial](#)
- 4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) [yes](#)
- 4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) [partial](#)
- 4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) [yes](#)
- 4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) [yes](#)
- 4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) [yes](#)
- 4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) [NA](#)
- 4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) [yes](#)